Retinal image classification

Natali Mercado Solórzano
Numéro d'étudiante : 20341195 Kaggle team: `ift6390_natali_mercado`

I. Introduction

A dataset of retinal images affected by Diabetic Retinopathy (DR) was provided. The goal of the project was to develop a machine learning model capable of classifying the severity of DR into five ordinal levels using these images.

To establish a baseline, a logistic regression model was trained on the original dataset after converting the images to grayscale and standardizing the features. The model was optimized using gradient descent, with tuning of both the learning rate and the number of iterations. Additional strategies were then explored, including data augmentation to mitigate class imbalance and dimensionality reduction to simplify the feature space. A class-weighted logistic regression model from scikit-learn was also tested.

Several additional models were evaluated, including Random Forest, Gradient Boosting, an SVM with an RBF kernel, and a small CNN.

II. Data exploration and preprocessing

A. *Data exploration*

A first exploration of the training dataset was performed. It contains (n = 1080) retinal images of shape ((28, 28, 3)) and (n = 1080) corresponding labels. Thus, the training set can be represented as

$$\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{n}, x_i \in \mathbb{R}^{28\times28\times3}, y_i \in \{0, 1, 2, 3, 4\},$$

where each y_i denotes the ordinal severity level of diabetic retinopathy.

To analyze the target distribution, the number of samples per severity level was counted. The results are summarized in the table below:

Severity Level	Number of Samples
0	486
1	128
2	206
3	194
4	66

This distribution clearly shows that the dataset is highly imbalanced, with class (0) being the majority class and class (4) extremely underrepresented. This imbalance motivated the later use of class weighting and data augmentation strategies during model development.

B. *Preprocessing*

*1) **Baseline and Scikit-Learn Models**:* The preprocessing pipeline applied to the baseline logistic regression model and the scikit-learn classifiers included a sequence of operations performed on both the training and test sets.

Gray scaling: The first step consisted in converting each RGB image to a grayscale representation by averaging the three color channels:

$$\text{gray}(x) = \frac{1}{3}(R + G + B).$$

This operation reduces each image from a $(28 \times 28 \times 3)$ tensor to a (28×28) matrix while preserving the structural patterns needed for classification.

Flattening: Once converted to grayscale, each image was reshaped into a vector. Given N images of size 28×28, the resulting representation lies in

$$\mathbb{R}^{784},$$

forming a matrix

$$X \in \mathbb{R}^{N\times784}.$$

Flattening was required because linear models such as logistic regression operate on one dimensional feature vectors.

Normalization: To improve numerical stability during optimization, pixel intensities were first scaled to the interval $[0, 1]$ by dividing by 255.

Feature-wise standardization was then applied:

$$X_{\text{norm}} = \frac{X - \mu}{\sigma},$$

where μ and σ correspond to the empirical mean and standard deviation of each feature computed from the training set. A small constant 10^{-8} was added to σ to avoid division by zero. For the test preprocessing, the mean and standard deviation computed on the training set were reused.

All sickit learn models used this approach through the train_preprocessing() function. In the graph below there is a comparison of images before and after processing

*2) **CNN**:* For the convolutional neural network, images were kept in tensor form rather than flattened. Although they were also converted to grayscale, the grayscale channel was duplicated to produce a $(28 \times 28 \times 3)$ tensor, matching the expected input shape of the network while preserving the spatial structure required by convolutional operations.

Below there is a summary of the preprocessing steps

Summary of Preprocessing Steps:

1) Load training and test data from pickle files.
2) Extract images and labels (training set only).
3) Convert RGB images to grayscale.
4) Flatten grayscale images into 784-dimensional feature vectors.
5) Normalize pixel intensities to $[0, 1]$.
6) Standardize features using the training-set mean and variance.
7) Apply the same normalization to the test set.

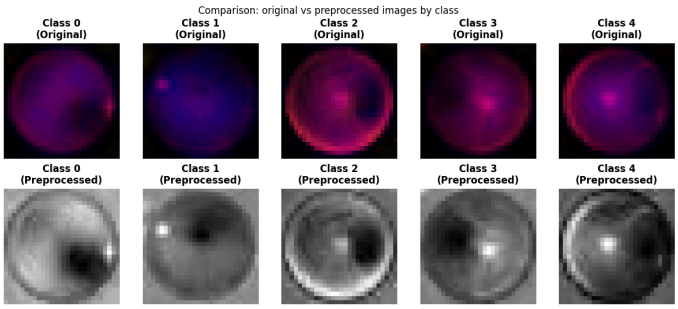

Figure 1. Preprocessed dataset.

C. *Logistic regression*

Logistic regression was implemented as a linear classifier operating on the flattened grayscale images, each represented as a vector in $\mathbb{R}^{784}$. Although the classes are not linearly separable in this space, the model serves as a baseline and allows a direct optimization of the cross-entropy loss.

A One-vs-Rest (OvR) scheme was used to handle the five-class problem. For each class $k \in \{0, 1, 2, 3, 4\}$, a binary target vector was constructed as

$$y_k^{(i)} = \begin{cases} 1, & \text{if } y^{(i)} = k, \\ 0, & \text{otherwise,} \end{cases}$$

and a separate logistic regression model was trained to predict $P(y = k \mid x)$.

Each classifier uses the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

with the input clipped to $[-500, 500]$ to prevent numerical overflow. The hypothesis for a sample x is

$$h_\theta(x) = \sigma(\theta^\top x),$$

where the parameter vector $\theta \in \mathbb{R}^{785}$ includes 784 weights and one bias term. The binary cross-entropy loss

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log\big(1 - h_\theta(x^{(i)})\big) \right]$$

was minimized using batch gradient descent. The gradient is given by

$$\nabla_\theta J(\theta) = \frac{1}{m} X^\top \left(\sigma(X\theta) - y \right),$$

and the update rule

$$\theta := \theta - \alpha \nabla_\theta J(\theta).$$

The implementation initializes all parameters to zero and augments the feature matrix with a column of ones to incorporate the bias term. Gradient descent proceeds until the norm of the gradient falls below 10^{-7}, and the loss is recorded at each iteration.

At inference time, all five classifiers compute their respective probabilities

$$p_k = \sigma(\theta_k^\top x),$$

and the predicted class is obtained by

$$\hat{y} = \arg\max_k \, p_k.$$

*1) **Experimentation**:* A grid search over learning rates $\alpha \in \{0.0005, 0.001, 0.005, 0.01\}$ and iteration counts $\{10{,}000, 20{,}000, 30{,}000\}$ identified the configuration $\alpha = 0.001$ with 20 000 iterations and tolerance 10^{-7} as the most effective.

A plot of the training and validation loss across epochs, shown below, illustrates the expected decay during optimization and indicates that the model is learning effectively without signs of overfitting.

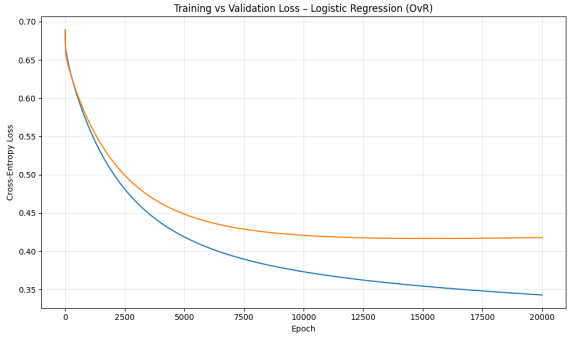

Figure 2. Training vs validation loss logistic regression model.

Nevertheless, the confusion matrix shown below indicates that the model does not predict class 4 at all, due to the low number of samples and the linear nature of the classifier.

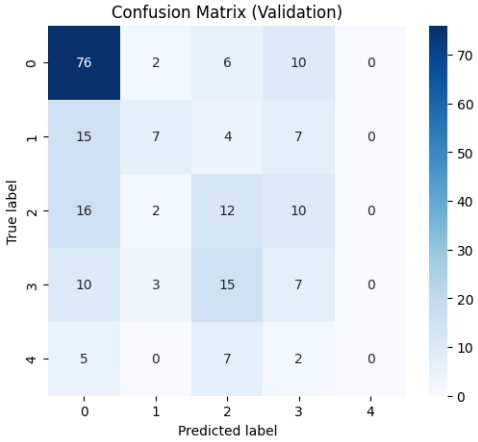

Figure 3. Training vs validation loss.

This model achieved approximately 48% validation accuracy and a 55.5% score on the private Kaggle leaderboard.

D. *Experimentation*

To address the misclassification of the minority classes, data augmentation was applied, increasing the dataset to a total of 3456 samples. The augmented dataset was then split into training and validation sets using a 20% validation ratio, after which a new hyperparameter search was performed. Using this configuration, the model was retrained with the best parameters identified, (learning rate of $\alpha = 0.0005$ and 20,000 iterations).

Although the validation accuracy decreased to 0.4220 (42.20%), the confusion matrix shows an improvement in class coverage: the model is able to predict at least one instance of class 4, a class that the original model failed to recognize entirely, as observed in the confusion matrix below.

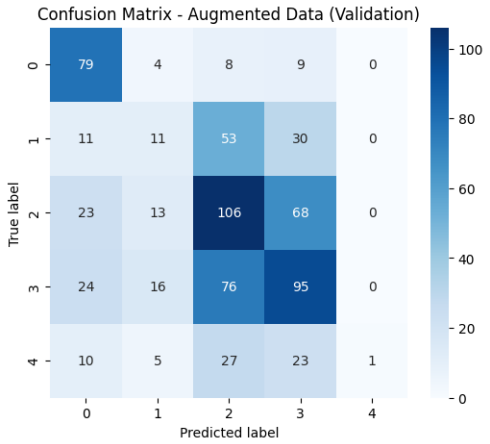

Figure 4. Training loss per class.

Then, PCA was applied with the objective to reduce dimensionality and computational cost. Different numbers of components (100, 200, 300, 400) were tested, but again the best model tuned on the validation data showed lower accuracy than the original one, with an accuracy of 0.4722 (47.22%).

After these unsuccessful attempts, sklearn's LogisticRegression with class weighting was explored. The class_weight='balanced' parameter automatically adjusts weights inversely proportional to class frequencies to handle the imbalance problem. Different regularization strengths (C = 0.1, 1.0, 10.0) were tested with L2 penalty and the 'lbfgs' solver. However, this approach also did not improve upon the baseline manual implementation.

Given the limitations of linear models, even with various preprocessing and balancing strategies, alternative non-linear models were explored.

E. *Scikit learn models*

With the data split into training and validation sets, four classical machine learning models were trained: Random Forest, Support Vector Machine (SVM) with a RBF kernel, Gradient Boosting, and k-Nearest Neighbors (KNN).

Random Forest is an ensemble method that constructs a collection of decision trees, each trained on a bootstrapped subset of the data. The final prediction is obtained by aggregating the individual tree outputs. This approach reduces variance and captures non-linear relationships, which often leads to strong performance on medium-sized datasets.

The SVM with a radial basis function (RBF) kernel maps the input features into a high-dimensional space where non-linear class boundaries can be represented through linear discriminant functions in the transformed space. This allows the model to learn complex separation surfaces even when the original feature representation is not linearly separable.

Gradient Boosting builds decision trees sequentially, with each new tree focusing on correcting the errors made by the previous ones. Although this produces a flexible and expressive model, it also makes Gradient Boosting more sensitive to noise and class imbalance. The implementation used $n\_estimators = 100$. Its performance was lower than that of Random Forest, likely because boosting gives greater importance to hard-to-classify examples, which in this dataset are predominantly associated with minority classes, leading to instability.

K-Nearest Neighbors classifies a sample based on the majority vote of its k nearest neighbors in the feature space; here, $k = 5$. While simple and non-parametric, KNN tends to perform poorly in high-dimensional settings due to the reduced discriminative power of distance metrics (the "curse of dimensionality"), which affects the flattened 784-dimensional image vectors used here.

Table I summarizes the validation accuracy and per-class accuracy obtained for each model.

Table I
VALIDATION ACCURACY AND PER-CLASS ACCURACY.

Model	Val	C0	C1	C2	C3	C4
RF	0.5231	0.872	0.152	0.275	0.400	0.071
GB	0.4537	0.809	0.121	0.200	0.200	0.214
SVM	0.4954	0.766	0.303	0.200	0.171	0.786
KNN	0.4722	0.809	0.121	0.350	0.229	0.000

The strongest models were Random Forest and the SVM with RBF kernel. This outcome is consistent with their ability to capture non-linear relationships.

Because Random Forest achieved the best validation accuracy, a dedicated hyperparameter tuning procedure was carried out for this model. The best configuration obtained was: {`'n_estimators': 200, 'max_depth': None, 'min_samples_split': 2`}. The tuned Random Forest also achieved the highest score on the public Kaggle leaderboard. However, the model reached perfect training accuracy, and the use of an unrestricted tree depth (`max_depth = None`) is a clear indication of overfitting. With no limit on depth, the trees grow until they reach full purity, effectively memorizing the training data instead of learning patterns that generalize well.

F. *CNN*

A custom convolutional neural network, SmallRGBNet, inspired by VGG-style architectures [1], was implemented to classify the 28×28 RGB images into the five categories. The model follows the VGG design principle of stacking 3×3 convolutions with ReLU activations and periodically applying 2×2 max pooling while increasing the number of channels as the spatial resolution decreases. The architecture consists of three sequential convolutional blocks. The first block applies a 3×3 convolution that transforms the input from 3 to 16

channels, followed by a ReLU nonlinearity and a 2×2 max-pooling operation that reduces the spatial resolution by a factor of two. The second block follows the same pattern, employing a 3×3 convolution that increases the feature dimensionality from 16 to 32 channels, again followed by ReLU activation and 2×2 max pooling. The third block performs a final 3×3 convolution, expanding the feature depth to 64 channels.

After the convolutional stages, an adaptive average pooling layer compresses each 64 channel feature map from 7×7 spatial resolution to a 1×1 representation, yielding a compact global feature descriptor. This tensor is then flattened and passed through a fully connected layer that produces the final output consisting of five class scores.

The same preprocessing as before pipeline was applied and the dataset was split into training and validation sets, with 20% reserved for validation. The model was trained for only 70 epochs. The resulting training and validation loss curves as well as the detailed metrics report are shown below.

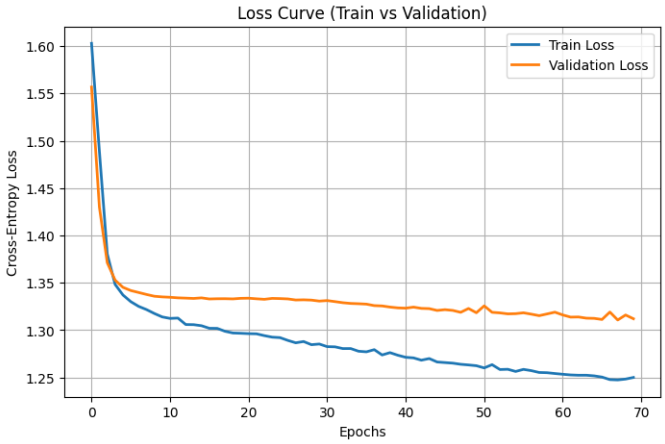

Figure 5. Loss curve.

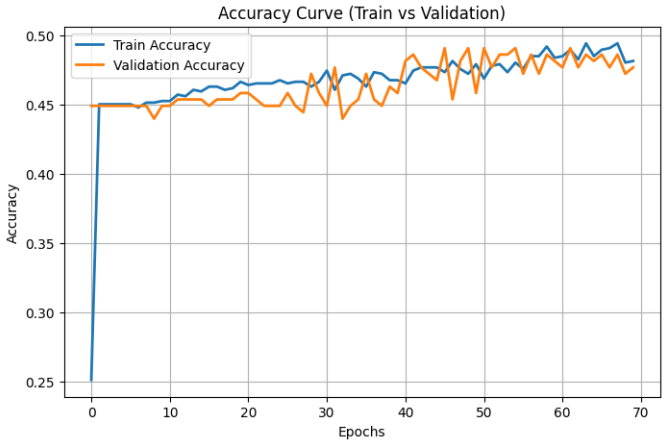

Figure 6. Accuracy curve.

The loss curves show that the model begins by learning effectively, with both training and validation loss decreasing rapidly during the first few epochs. However, after this initial phase, validation loss stabilizes while training loss continues to decline. This indicates that the model keeps improving on the training data but does not generalize much further to unseen data.

Table II
CLASSIFICATION REPORT

	Precision	Recall	F1-score	Support
0	0.56	0.85	0.67	97
1	0.25	0.04	0.07	26
2	0.21	0.17	0.19	41
3	0.42	0.33	0.37	39
4	0.00	0.00	0.00	13
Accuracy			0.48	216
Macro avg	0.29	0.28	0.26	216
Weighted avg	0.40	0.48	0.41	216

The learning process is therefore somewhat limited: the model does not diverge, but it also does not manage to significantly reduce the gap between its performance on training and validation data.

In the accuracy curves, the training and validation accuracies remain close throughout the 60 epochs, fluctuating around the 0.45–0.49 range. The fact that both curves follow each other so closely indicates that the model is not suffering from severe overfitting

Analyzing the classification report in TABLE II, it becomes clear that the model struggles considerably with the minority classes. Class 4, in particular, is never predicted, as reflected by precision, recall, and F1-score values of zero. Although precision is relatively acceptable for classes 0 and 3, the recall for most classes especially 1, 2, and 4 is very low, indicating that the model fails to correctly identify a substantial portion of these samples which suggests that the model is heavily biased toward the majority class and overlooks the less represented classes. This behavior is not surprising given that no over-sampling or data balancing techniques were applied, meaning the training set remained highly imbalanced. Additionally, no hyperparameter tuning was performed, which further limits the model's ability to generalize effectively across all classes.

G. *Conclusion*

Several machine learning algorithms were explored to classify the provided retinal image dataset. A baseline logistic regression model was first implemented, along with different preprocessing, data augmentation, and dimensionality-reduction strategies. Although these variations did not lead to major improvements, they helped highlight the challenges caused by the strong class imbalance present in the dataset.

Non-linear models from scikit-learn, including Random Forest, Gradient Boosting, SVM with an RBF kernel, and KNN, were then evaluated. Among them, Random Forest achieved the highest validation accuracy, but it also showed clear overfitting due to its tendency to memorize the training data. In contrast, logistic regression, although less accurate overall, demonstrated more stable generalization.

A small convolutional neural network was finally trained to better capture spatial information in the images. The CNN showed initial learning progress, but its validation performance plateaued early, and the model struggled with minority classes. The classification report confirmed that class imbalance remained a major obstacle: the network rarely recognized classes 1 and 2 and failed to identify class 4 at all. Nevertheless, the curves showed the expected behavior of learning.

The logistic regression model implemented from scratch ultimately achieved the highest score on the Kaggle private leaderboard, alongside the vgg inspired CNN with small depth. This result reflects the limitations imposed by the dataset: severe class imbalance, a small number of training samples, and low-resolution images restricted the amount of meaningful structure any model could learn. In such conditions, simpler models can sometimes generalize better, whereas deeper architectures are unable to take advantage of their capacity.

Overall, the experiments indicate that the main bottleneck lies not in model choice but in the quality of the input features. The images appear heavily transformed and low-resolution, with preprocessing steps that may have distorted relevant retinal patterns. This substantially limits the discriminative information available to all classifiers. Consequently, improving data quality through access to higher-resolution images or feature extractors tailored to medical imaging would likely yield greater gains than increasing model complexity.

REFERENCES

[1] K. Simonyan and A. Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, arXiv:1409.1556, 2014. doi:10.48550/arXiv.1409.1556